
Can Agent Benchmarks Support Their Scores?

Evidence-Supported Bounds for Interactive-Agent Evaluation

Anonymous Author(s)

Affiliation

Address

email

Abstract

Interactive-agent benchmarks map an agent run to a binary outcome via outcome checks. When checks rely on surface-level signals or miss the agent’s action path, they fail to determine whether the run succeeded. For example, a benchmark case asks whether Alice’s shipping address changed, but its outcome check only verifies that the agent clicked Save. This does not guarantee that the intended state change was applied; the agent may have updated the wrong record. Counting such a run as a success makes the score misleading. Benchmark quality therefore depends on outcome detection as well as task design.

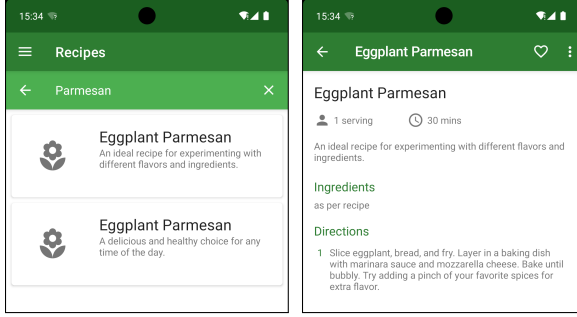
We address this by adding an outcome-evidence reporting layer to existing benchmarks, without changing their tasks, agents, or evaluators. The layer does three things: (i) specifies, before scoring, which stored artifacts would verify the claimed outcome for each case; (ii) applies the locked case checklist to each completed record and assigns one of three evidence labels: Evidence Pass, Evidence Fail, or Unknown; and (iii) reports evidence-supported bounds that quantify uncertainty from Unknown records. Unknown records are kept visible rather than silently counted, dropped, or hidden inside a single success rate.

We evaluate this outcome-evidence layer on five public benchmarks: ANDROID-WORLD, AGENTDOJO, APPWORLD, τ^3 -bench retail, and MINIWOB. The resulting reports make outcome-evidence quality visible alongside task quality, supporting more evidence-grounded agent evaluation and uncertainty reporting. We release the case checklists, reason labels, audit checks, robust-ranking summaries, and scoring script needed to reproduce the reports.

1 Introduction

Interactive-agent benchmarks increasingly ask agents to navigate interfaces, call tools, and change persistent environment state. Their headline result is usually a single success rate, but a reported success is only as reliable as the outcome check behind it. Figure 1 shows a simple example from the released ANDROIDWORLD benchmark: the task asks an agent to delete recipes whose directions contain Parmesan; the trace shows the agent opening visible Eggplant Parmesan recipes and deleting nothing; yet the benchmark still reports success. This kind of mismatch can arise when an evaluator is too weak, when a benchmark has a bug, or when the retained artifacts are insufficient to verify what happened.

This is an outcome-evidence gap. The benchmark claim is a binary statement about what happened in the environment, while the stored artifacts may include only screenshots, action logs, final messages, partial verifier inputs, or other indirect traces. The gap is especially important for interactive agents because correctness often depends on identity resolution, side effects, post-state queries, policy constraints, and multi-step tool interactions. When these artifacts are insufficient, a point score can overstate what the benchmark has actually verified. In an ideal benchmark whose claims, evaluators,



Screen 1: searched for Parmesan Screen 2: opened recipe details

Figure 1: One case from ANDROIDWORLD (source). The task asks the agent to delete recipes whose directions contain Parmesan. The retained screens show the key trace: the agent (i) searches for Parmesan, (ii) opens an Eggplant Parmesan recipe, and (iii) deletes nothing. The run is nevertheless reported as successful because the released target construction leaves the evaluator with an empty target set, while recipes treated as non-targets mention Parmesan. This creates a false success label: the benchmark output says the deletion succeeded.

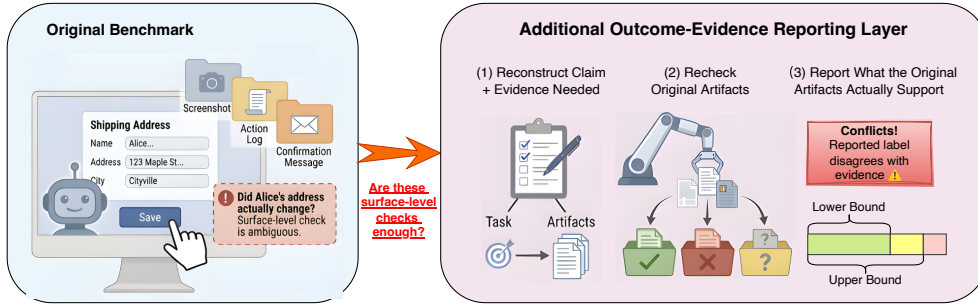


Figure 2: Overview of the outcome-evidence gap and our reporting layer. A benchmark may report success from surface artifacts even when those artifacts do not verify the intended state change. Our layer leaves the benchmark run unchanged, then states the claim and needed evidence, rechecks the benchmark’s saved artifacts, and reports what the original artifacts actually support: evidence labels, label conflicts, and evidence-supported bounds.

38 and retained state are fully aligned, every completed run would be decidable. Unknown records
39 appear when the stored artifacts are insufficient to verify the benchmark’s own outcome claim.

40 We address this gap with an outcome-evidence reporting layer. The layer is additive: it leaves
41 benchmark tasks, agents, environments, and native evaluators unchanged. For each case, it records a
42 short case checklist before scoring, specifying which stored artifacts would decide the benchmark’s
43 own success claim. Each completed record is then labeled Evidence Pass, Evidence Fail, or Unknown.
44 Using P , F , U for the three counts, the report keeps Unknown records visible rather than forcing
45 them into success, failure, or exclusion.

46 When a record is Unknown, none of the usual shortcuts is neutral. Counting it as a success overstates
47 what the evidence supports. Counting it as a failure can punish missing instrumentation rather than
48 agent behavior. Dropping it changes which records are evaluated and can create agent-dependent post-
49 selection, because some agents leave more decisive traces than others. We therefore keep Unknown
50 records in the same set of completed records and report the range of all-record performance still
51 compatible with the stored evidence. The counted-only score is still useful, but only as a diagnostic
52 conditional on decidability, not as a headline all-record estimate.

53 We evaluate this reporting layer on five public interactive-agent benchmarks. ANDROIDWORLD,
54 AGENTDOJO, APPWORLD, τ^3 -bench retail, and MINIWOB span mobile UI tasks, paired security
55 tasks, stateful API tasks, retail tool use, and web UI microtasks. We do not assume in advance which
56 benchmark is easy or hard to verify. Instead, the study is designed to be falsifiable: if benchmarks
57 with rich stored state still produce many Unknown records, our evidence requirements may be
58 too strict or the retained artifacts may be weaker than expected; if uncertainty concentrates where
59 outcome claims depend on missing state, identity mappings, or side effects, the layer has found a real
60 outcome-evidence gap.

The paper makes four contributions: (i) we formalize the outcome-evidence gap in interactive-agent benchmarks, where a binary success claim may be reported even when stored artifacts are insufficient to decide that claim; (ii) we introduce case checklists tied to each benchmark’s own success claim, plus a three-state counting rule that separates Evidence Pass, Evidence Fail, and Unknown records without changing benchmark tasks, agents, environments, or native evaluators; (iii) we derive evidence-supported performance bounds over the fixed set of completed records, making the counted-only score a diagnostic over decidable records rather than a headline all-record estimate; and (iv) we apply the method to five public benchmarks with different task and evaluator designs, report where uncertainty and label conflicts arise, and release the case checklists, reason labels, audit checks, robust-ranking summaries, and scoring script needed to reproduce the reports.

2 Related Work and Preliminaries

Interactive-agent evaluation and outcome checks. Interactive-agent benchmarks evaluate agents that navigate interfaces, call tools, and alter environment state. Web-agent benchmarks include MINI-WoB [10], WebShop [28], Mind2Web [3], VisualWebArena [9], WebArena [31], and BrowserGym [1]. Tool-use and stateful-interaction benchmarks include ToolBench-style tool-use benchmarks [19], τ -bench [29], τ^3 -bench retail [22], and ToolSandbox [11]. Broader agent environments include AGENTDOJO [2], ANDROIDWORLD [20], WORKARENA [4], OSWORLD-VERIFIED [25], and long-horizon workplace settings such as TheAgentCompany [26]. These benchmarks make interactive evaluation realistic, but realism alone does not guarantee that a stored trace decides the outcome claim being reported.

Verified evaluators and benchmark validity. A recent line of work strengthens outcome checks through state-based or verified evaluation, including the verified WEBARENA-VERIFIED release [7], APPWORLD database-state unit tests [23], and SWE-bench Verified [16]. Benchmark-validity work, including the Agentic Benchmark Checklist, argues that construct validity, outcome validity, and reporting assumptions should be made explicit [32]. Our contribution is complementary: rather than proposing a new environment, a stronger domain evaluator, or an audit checklist, we add a cross-benchmark reporting layer that asks what the stored artifacts of completed runs can support under each benchmark’s own claim. LLM-as-judge and trajectory-evaluation work [30, 21, 12], benchmark refresh or contamination studies [8, 6, 27], and empirical audits of benchmark solutions [24, 17] are useful diagnostics, but they do not replace evidence that a native outcome claim is decidable from stored artifacts. Dataset and model reporting work similarly argues that evaluation artifacts should make their assumptions visible [5, 15, 13, 18]; our unit is a completed interactive-agent record and the artifacts retained to verify its outcome claim.

Claims, records, traces, and evidence. A *case* specifies a task, the initial environment, the benchmark claim being tested, and the evidence required to decide that claim. A benchmark claim is the binary outcome statement the benchmark reports for a completed run, such as whether a target record was updated, a message was sent, or a constraint was violated. The *native evaluator* is the benchmark’s released mechanism for turning its available artifacts into a success or failure label.

We use *case unit* only for a small bookkeeping issue: in most domains, one task instance is one case unit, while in paired-arm settings such as AGENTDOJO, two runnable arms form one case unit even though they are executed separately. Decisions live at the level of *records*. A record is one agent’s result on one case unit. The distinction matters because the same case unit can be countable for one agent, whose trace exposes decisive evidence, and Unknown for another, whose trace does not.

An *episode* is one concrete execution of one agent on one runnable arm of a case. Episodes produce *traces*: ordered records of actions, observations, tool calls, browser events, messages, files, and post-run artifacts. We use *stored artifacts* for the full set of traces, logs, state snapshots, receipts, verifier inputs, and post-run measurements retained after execution. *Evidence* is the subset of those artifacts that can support the active benchmark claim. A trace can be plausible without being decisive.

Staying tied to the benchmark claim. Because the method sits on top of existing benchmarks, the requirements must not silently make the task harder. For each case unit, we write down: (i) the benchmark’s own binary claim; (ii) the official source of that claim, such as the task text, evaluator

code, policy, or schema; (iii) the artifacts needed to decide success or failure; (iv) the rule for assigning Unknown; and (v) whether any requirement goes beyond the benchmark’s own claim.

The requirements must be minimal with respect to the benchmark’s own semantics. For example, if a task says only “update the address” and the native evaluator only checks the target record’s address, then the requirements ask for target-record identity and post-state evidence. A no-collateral-change requirement is included only if the benchmark’s own task, policy, evaluator, or reported claim requires it. Otherwise it is labeled as a stronger-measurement claim and reported separately.

When sources differ, the requirements follow a fixed source hierarchy: official evaluator semantics first; official task text and policy second; and schema constraints needed to interpret evaluator-visible state third. We do not add requirements from annotator intuition alone. Any requirement not grounded in one of these sources is labeled as a stronger-measurement claim and excluded from the main bounds tied to the benchmark’s own claim.

3 Method

The method turns completed benchmark runs into an evidence-supported report. It keeps the benchmark’s native view, namely the released evaluator’s pass/fail label, and adds an evidence view that asks whether the stored artifacts are sufficient to decide the same underlying outcome claim. The layer is post-run with respect to agent execution but fixed before evidence scoring: it does not change the task, prompt, agent, environment, native evaluator, or native score.

What Unknown means. Unknown is not a third kind of agent behavior. A record receives the Unknown label when the available artifacts do not decide whether the benchmark’s own success claim is true. A clicked Save button, a final confirmation message, or a passed proxy check may be insufficient if the retained state does not show that the intended record changed. Conversely, timeouts, invalid actions, tool misuse, and aborts after task start are failures when the native benchmark would fail them; they are not moved into Unknown. In an ideal state-based benchmark with explicit claims, aligned evaluators, complete retained state, and unique entity mappings, Unknown records should be rare or absent. Their presence measures an evidence gap in the benchmark report, not task difficulty or agent uncertainty.

Case checklist. For each case unit, we first make the outcome claim explicit. An LLM-assisted drafter reads the user goal, task text, official policy when present, released evaluator or oracle code, and relevant state schemas. It proposes a compact case checklist: what the user is trying to achieve, what condition the benchmark reports as success, which official code or oracle checks that condition, which artifacts would decide it after execution, and when the record should be Unknown. For every checklist field filled by the LLM-assisted drafter, the checklist must include either a rationale or a source pointer identifying the file and location from which the information was derived. Human reviewers source-check each checklist against the hierarchy in Section 2. The checklist is not a new task or answer key; it states what stored evidence would be sufficient to verify the benchmark’s own claim. In addition, the LLM-assisted drafter may propose conditions that would require a stronger-measurement claim, together with a brief rationale for why those conditions go beyond the benchmark’s native success criterion. Human reviewers adjudicate these proposed conditions, and we report them separately from the native-aligned result.

Evidence scoring. We then run the benchmark normally and retain the artifacts available for review: traces, tool calls, evaluator inputs and outputs, final messages, and saved database, browser, file, or API state. For each completed record, the native view records the released evaluator’s label. The evidence view applies the locked case checklist to the stored artifacts and assigns one of three labels: Evidence Pass, if the artifacts support success under the benchmark claim; Evidence Fail, if they support failure; and Unknown, if they do not decide the claim. For every judgment made or proposed by an LLM-assisted scorer, the record must include either a rationale or a source pointer identifying the file and location from which the information was derived. Thus the evidence view is an additional report over the same runs, not a replacement for the native benchmark score.

Human review and correction. The LLM-assisted scores are provisional until reviewed. Before computing the final aggregate counts, human reviewers inspect records flagged by a native/evidence

disagreement, an Unknown label, a stronger-measurement downgrade, or a sampled audit trigger. Reviewers classify each flagged record as a benchmark/evaluator issue, an evidence gap, a stronger-only issue, a scorer/checklist mismatch, or pending adjudication. Scorer/checklist mismatches are corrected before the final reported counts are generated. Benchmark and evaluator issues remain in the results as findings about what the original score supports; stronger-only issues stay outside the native-aligned score. We release the item-level review ledger and a benchmark-level audit summary so the amount of human correction is visible rather than hidden inside the final numbers.

Counting rule. The reporting rule uses only the three evidence-state counts. For agent a in domain d , let $N_{a,d}$ be the number of completed records in the evaluation set, and let $P_{a,d}$, $F_{a,d}$, and $U_{a,d}$ be the counts of Evidence Pass, Evidence Fail, and Unknown records, so $N_{a,d} = P_{a,d} + F_{a,d} + U_{a,d}$. The counted-only score answers a conditional question: among records the stored artifacts decide, what fraction are successes? The bound answers the all-record question: over every completed record, what performance values are still compatible with the stored evidence? Writing P, F, U, N for the counts in a given agent-domain cell, we report

$$\text{CountedScore}(a, d) = \frac{P}{P + F}, \quad \text{Perf}(a, d) \in \left[\frac{P}{N}, \frac{P + U}{N} \right], \quad \text{Width}(a, d) = \frac{U}{N}.$$

The lower endpoint counts only Evidence Pass records; the upper endpoint treats every Unknown record as a possible success. The width is exactly the Unknown share. When $U = 0$, the bound collapses to the all-record score. When $U > 0$, a single point estimate must make an unobserved decision about records the artifacts did not decide. These intervals are not confidence intervals; they are partial-identification bounds describing what the stored evidence can and cannot identify [14]. The counted-only score remains useful as a diagnostic conditional on decidability, but the headline object is the evidence-supported bound.

Comparisons and diagnostics. We keep three diagnostics separate from the headline bounds. First, case-cluster bootstrap intervals describe sensitivity to the finite set of sampled cases. Each replicate resamples case units within a domain and keeps all agent records attached to the resampled case unit. These intervals condition on the stored traces and fixed agent versions; they do not measure uncertainty from repeated executions, API drift, tool nondeterminism, or environment volatility.

Second, the same bounds determine what leaderboard claims the benchmark can support. A benchmark can still print a point-score ordering when bounds overlap, but that ordering is not identified by the stored evidence: Unknown records could change the direction or erase the gap. We therefore treat pairwise separation as a benchmark-usefulness diagnostic. If many model pairs are marked “?”, the benchmark may still be useful for finding failures, but it is weak evidence for a strict leaderboard. Formally, for agents i and j in domain d , $\Delta(i, j; d) = [\text{LB}_{i,d} - \text{UB}_{j,d}, \text{UB}_{i,d} - \text{LB}_{j,d}]$. We report $i \succ j$ only if $\text{LB}_{i,d} > \text{UB}_{j,d}$, and $j \succ i$ only if $\text{LB}_{j,d} > \text{UB}_{i,d}$; otherwise the pair is marked “?” rather than treated as a tie. For a separated ordering $i \succ j$, the margin is $M_{i \succ j} = \text{LB}_{i,d} - \text{UB}_{j,d}$. An asterisk means the 2.5th percentile of the case-cluster bootstrap distribution of $M_{i \succ j}$ remains positive; it is a descriptive stability tag, not a significance test.

Third, every Unknown record receives one reason, so the bound width can be audited. The reasons are: missing state query (needed backend or environment state was not saved), unobservable side effect (the consequence of an action is not exposed), ambiguous identity mapping (the acted-on record or recipient is not unique), missing state-preservation evidence (the native claim requires unrelated state to remain unchanged but no diff or snapshot is available), paired-arm asymmetry (one arm of a paired case is decidable and the other is not), evaluator-output ambiguity (the released evaluator emits a label but the stored inputs do not determine it), and claim-scope mismatch (the trace supports a related task but not the reported claim). Appendix C gives representative examples and the priority rule used when more than one reason could apply.

4 Experiments

We evaluate whether the reporting layer changes what benchmark scores can claim, not which model wins a leaderboard. The experiment fixes five public interactive-agent benchmarks, three models, and a 100-case sample per benchmark. Each run is scored twice: once by the benchmark’s native evaluator and once by the evidence-supported reporting layer.

Table 1: Benchmark suite. Each benchmark contributes 100 case units and is run with three models. The suite spans different task types, native evaluators, and stored-artifact regimes.

Benchmark	Task/evaluator setting	Native check
ANDROIDWORLD	Mobile UI tasks	Android app task evaluator.
τ^3 -bench retail	Retail tool use	Retail-state success signal.
AGENTDOJO	Paired utility/security tasks	Utility/security labels over paired arms.
APPWORLD	Stateful API tasks	Database-state unit tests.
MINIWOB	Web UI microtasks	DOM/task-specific success checks.

Benchmark suite. The suite spans five evaluator and artifact regimes: mobile UI tasks, paired utility/security labels, stateful API tasks, retail tool-use state, and web UI microtasks. We do not assume in advance that any benchmark should be easy to verify. The test is whether the layer stays narrow when retained artifacts decide the claim, and exposes uncertainty only when they do not. Across domains, the evidence view uses artifacts retained by the normal benchmark run: traces, tool calls, evaluator inputs and outputs, final messages, and saved database, browser, file, or API state when available.

Models and workload. We run OpenAI GPT-5.4, Claude Opus 4.7, and DeepSeek V4 Pro through OpenRouter. All three use temperature 0, a 4096-token output budget, a 120-second timeout, and two retries. The paper does not treat these models as an exhaustive comparison set. They are fixed measurement probes used to test whether outcome-evidence uncertainty is a property of benchmark evidence rather than an artifact of one model family. The release records concrete model identifiers, prompts, tool wrappers, run dates, and API settings.

For each benchmark, the manifest selects 100 eligible case units using a deterministic hash order after excluding smoke-test and malformed cases. Each case unit is run once per model, giving $5 \times 100 \times 3 = 1500$ completed record slots before infrastructure exclusions. AGENTDOJO executes two arms per record, so the main experiment stores 1800 execution episodes in total: 600 for AGENTDOJO and 300 for each of the other four benchmarks.

Execution and evidence scoring. Every model run uses the benchmark’s original task interface, tools, environment, and released evaluator. Evidence collection is handled by benchmark-specific adapters that save raw run records, artifact manifests, native evaluator outputs, traces, logs, and available post-run state. The adapters do not make final evidence-label decisions, so the native score remains directly recoverable from the saved artifacts.

Before evidence scoring, an LLM-assisted drafter produces one case checklist per case unit from the task text, official policy or evaluator when present, state schemas, and retained artifact schema. Each LLM-filled field must include a rationale or source pointer. Two researchers cross-validate the checklists in two rounds; unsupported items are edited or regenerated and reviewed again. Once locked, checklists cannot be changed based on agent outcomes. Proposed stronger-measurement conditions are adjudicated separately and scored outside the native-aligned result.

Initial evidence scoring then applies the locked checklist to each saved evidence directory with a read-only LLM-assisted scorer. The score bundle stores the label, rationale/source pointers, logs, telemetry, event stream, and prompt/schema hashes so every Evidence Pass, Evidence Fail, or Unknown judgment can be audited. Exact model-call configuration and reproduction commands appear in Appendix F.

The final evidence table is produced only after human adjudication of flagged records. The adjudication ledger uses the same schema for every benchmark: case id, model id, layer, trigger, current labels, human audit label, recommended correction, final label fields, evidence summary, and source pointers. This makes the process extensible as additional benchmark results arrive, and prevents LLM-scoring mistakes from being silently folded into the reported aggregate.

Denominator rule. Only infrastructure or pre-run failures are excluded from the evaluation set. Examples include a sandbox that fails to boot before the task is presented, a missing benchmark asset, or an external service outage that prevents normal benchmark execution. Agent-caused invalid

Table 2: Main measurement report. Counts are over completed records after excluding only infrastructure/pre-run failures. Native score is the released benchmark score on the same completed records. Bound is the all-record evidence-supported interval [Lower, Upper]; Width is the Unknown share. Gap is the signed distance from the released score to the nearest endpoint when the released score falls outside the bound.

Benchmark	N	Native score	Evidence counts $P/F/U$	Bound	Width	Gap
ANDROIDWORLD	300	FILL	FILL/FILL/FILL	[FILL, FILL]	FILL	FILL
τ^3 -bench retail	300	77.0%	212/87/1	[70.7%, 71.0%]	0.3%	+6.0 pp
APPWORLD	300	FILL	FILL/FILL/FILL	[FILL, FILL]	FILL	FILL
AGENTDOJO	300	FILL	FILL/FILL/FILL	[FILL, FILL]	FILL	FILL
MINIWOB	300	FILL	FILL/FILL/FILL	[FILL, FILL]	FILL	FILL

actions, tool misuse, timeouts after task start, malformed final answers, or benchmark-facing aborts are not excluded. They are counted as failures when the native benchmark would fail the run, and they are also reported in the all-attempt audit in Table 5. This separation prevents evidence filtering from raising scores by removing difficult or agent-induced failures.

Sample size and precision. The primary unit for uncertainty is the case unit, not the record. Each benchmark has 100 case units crossed with three models, and the three records attached to a case unit can be correlated because the models face the same task. We therefore avoid formal precision claims based on 300 independent records per benchmark. A record-level binomial calculation would be, at best, an independence lower bound and is not used for inference. All resampling intervals are case-clustered and condition on the stored traces.

Human audit and rerun checks. Two researchers audit records triggered by native/evidence disagreement, Unknown labels, stronger-measurement downgrades, and sampled audit checks. Auditors inspect the task, released evaluator output, trace, artifacts, checklist, and score rationale, then mark whether the label stands, needs a scorer/checklist correction, reveals a benchmark/evaluator issue, or remains pending. Aggregate result tables use corrected labels, while the audit summary reports how much correction was needed. We also rerun OpenAI GPT-5.4 on ten case units per benchmark as a small repeated-execution probe; this is not used as a model-performance uncertainty interval.

5 Evaluation

The evaluation asks whether a benchmark report can support the claims implied by its own scores. The point is not to replace native evaluators with a stricter judge. It is to make three distinctions visible: whether the stored artifacts decide the native claim, whether the native label agrees with that evidence, and whether a leaderboard ordering remains identified after Unknown records are kept in the analysis. We therefore treat Unknown as an evidence property of the benchmark record, not as an agent error, and we keep stronger-measurement conditions separate from the native-aligned result.

Score support. Table 2 is the central measurement table. It keeps the released native score visible, then reports the evidence counts $P/F/U$, the evidence-supported bound, its width, and the signed gap when the released score falls outside the bound. The interpretation is direct: a narrow bound containing the native score means the stored artifacts support the reported score. A narrow bound excluding the native score points to an evaluator-label mismatch, because the artifacts decide the claim differently from the released evaluator. A wide bound means the artifacts are insufficient to decide enough records, so the benchmark may still be valuable for execution but cannot support a precise all-record score from the retained evidence alone. The counted-only score is omitted from the main table because it conditions on decidable records; Appendix D reports it as a diagnostic.

Diagnosing unsupported scores. The same bound width can hide different benchmark-design problems, so the next step is diagnosis rather than another score. Some unsupported scores come from missing retained evidence: the run may have changed a backend state, sent a message, or preserved unrelated state, but the released artifacts do not expose the value needed to check the claim. Other

Table 3: Benchmark-facing diagnosis. Table 2 reports the score and bound magnitudes; this table summarizes the main accepted audit finding for each benchmark and the repair suggested by that finding.

Benchmark	Dominant accepted issue	Benchmark-facing repair
ANDROIDWORLD	FILL	FILL
τ^3 -bench retail	Reward/action mismatch	Align scalar reward with action/state criteria; retain authoritative post-run state.
APPWORLD	FILL	FILL
AGENTDOJO	FILL	FILL
MINIWOB	FILL	FILL

Table 4: Ranking resolution under evidence-supported bounds. Each benchmark has three pairwise model comparisons. A pair is identified only when the two evidence-supported bounds are separated; bound overlap means the stored evidence does not support a directional ranking for that pair.

Benchmark	Identified pairs	Not identified	Leaderboard implication
ANDROIDWORLD	FILL/3	FILL/3	FILL
τ^3 -bench retail	3/3	0/3	All three model pairs are separated by evidence-supported bounds in the current sample.
APPWORLD	FILL/3	FILL/3	FILL
AGENTDOJO	FILL/3	FILL/3	FILL
MINIWOB	FILL/3	FILL/3	FILL
All benchmarks	FILL/15	FILL/15	Unsupported native strict orderings: FILL; native conflicts: FILL.

292 unsupported scores come from a direct contradiction: the retained artifacts decide the claim, but not
 293 in the direction reported by the native evaluator. These two cases require different repairs. Table 3
 294 therefore reports the dominant accepted audit finding and the corresponding benchmark-facing repair
 295 for each benchmark, instead of repeating the magnitudes already shown in Table 2. Full reason
 296 histograms, counted-only scores, and case-resampling intervals appear in Appendix D.

297 **Leaderboard resolution.** A single success rate is often used to rank agents, but an evidence-
 298 supported report can only rank two agents when their bounds are separated. With three models,
 299 each benchmark yields three pairwise comparisons. If one model’s lower endpoint exceeds another
 300 model’s upper endpoint, the stored artifacts support a directional ranking for that pair. If the bounds
 301 overlap, the result is not a tie; it is a loss of leaderboard resolution, because different completions
 302 of the Unknown records can reverse or erase the apparent ordering. Table 4 reports this directly.
 303 This is the comparison that matters for benchmark usefulness: a benchmark may still produce a
 304 point leaderboard, but if many pairs are not identified, the stored evidence does not support the
 305 leaderboard’s strict order.

306 **Human audit.** The reporting layer itself must be inspectable. Because checklist drafting and record
 307 scoring are LLM-assisted, final tables use labels corrected after two-researcher audit. The audit ledger
 308 separates kept LLM-assisted judgments, scorer/checklist corrections, and substantive benchmark-
 309 facing findings; the aggregate and per-benchmark counts appear in Table 9 as quality-control metadata
 310 rather than another performance score.

311 6 Discussion

312 The main empirical claim is not that evidence-supported reporting must lower benchmark scores.
 313 It can lower a score, leave it unchanged, or expose only a small residual uncertainty. The sharper
 314 claim is conditional: when retained artifacts decide the native outcome, the evidence bound should
 315 be narrow and should either contain the native score or reveal a concrete evaluator mismatch; when
 316 artifacts do not decide the outcome, the bound should widen and the reason labels should identify
 317 what evidence is missing. This is why Table 2 and Table 3 should be read together. The first table

318 says how much of the score is supported; the second says what benchmark authors would need to fix
319 to make the unsupported part decidable.

320 Unknown records are not an inherent property of interactive-agent evaluation. They arise when the
321 benchmark’s stored artifacts do not decide the benchmark’s own claim. A perfectly instrumented
322 benchmark can have zero Unknown records: for example, a state-based task with a precise claim,
323 retained pre/post state, unique object identifiers, and evaluator inputs sufficient to reproduce the
324 decision should make every completed record decidable. Conversely, a realistic interactive task can
325 remain valuable while still producing Unknown records if the released trace omits the state needed
326 to verify side effects, identity, or state preservation. The distinction matters because it changes the
327 remedy. The agent need not be penalized for missing evidence, and the benchmark need not be
328 discarded; the report should say that the score is only partially supported by the artifacts retained.

329 The ranking result is the same point applied to leaderboards. Point scores can create an apparent
330 ordering even when the underlying evidence supports more than one ordering after Unknown records
331 are completed. Table 4 therefore asks a stricter but more decision-relevant question: how many model
332 pairs are actually separated by the evidence-supported bounds? A non-identified pair is not a tie and
333 not a statistical insignificance claim. It says that the stored evidence does not support choosing a
334 directional winner for that pair. This is the setting where a benchmark can be useful for qualitative
335 debugging yet weak as a leaderboard instrument.

336 The native-aligned layer and the stronger-measurement layer also serve different purposes. The native-
337 aligned layer asks what the original benchmark score can support under the benchmark’s own claim.
338 Stronger-measurement conditions ask whether the task would remain successful under additional
339 requirements that benchmark users may care about, such as exact recipient identity, complete side
340 effects, or preservation of unrelated state. Keeping these layers separate avoids moving the goalposts:
341 native failures are benchmark-score support problems, while stronger-measurement failures are
342 evidence about what the original benchmark did not claim to measure.

343 The main limitations follow from this design. A vague native claim cannot be fully repaired by
344 a cleaner report; the reason taxonomy is a study vocabulary rather than a universal ontology; the
345 three evaluated models are measurement probes rather than a definitive leaderboard; and the case-
346 resampling intervals condition on stored traces, fixed agent versions, and fixed benchmark artifacts.
347 Human adjudication is therefore part of the method, not an afterthought: LLM-assisted checklists and
348 labels must carry source pointers, flagged cases must be audited, and scorer/checklist corrections must
349 be separated from benchmark-facing findings. The practical standard is simple: future benchmark
350 reports should expose the counts, bounds, reason labels, and audit ledger needed to see whether the
351 reported score is actually supported.

352 Interactive-agent benchmarks should not have to choose between a single score and an unstructured
353 audit trail. We introduced a reporting layer that keeps the native benchmark intact but asks what
354 its stored artifacts can verify. The result is a score report with Evidence Pass, Evidence Fail,
355 Unknown, and evidence-supported bounds; a diagnosis of why any unsupported portion remains; and
356 a leaderboard check that separates identified rankings from unidentified comparisons. The broader
357 lesson is that benchmark quality depends not only on task design and agent difficulty, but also on
358 whether the evidence retained after execution can support the score being reported.

References

- [1] Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. The browsergym ecosystem for web agent research, 2024. URL <https://arxiv.org/abs/2412.05467>.
- [2] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024. URL <https://arxiv.org/abs/2406.13352>.
- [3] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.
- [4] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks?, 2024. URL <https://arxiv.org/abs/2403.07718>.
- [5] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723. URL <https://cacm.acm.org/research/datasheets-for-datasets/>.
- [6] Batu Guan, Xiao Wu, Yuanyuan Yuan, and Shaohua Li. Is your benchmark (still) useful? dynamic benchmarking for code language models, 2025. URL <https://arxiv.org/abs/2503.06643>.
- [7] Amine El Hattami, Megh Thakkar, Nicolas Chapados, and Christopher Pal. Webarena verified: Reliable evaluation for web agents. SEA @ NeurIPS 2025 poster, 2025. URL <https://openreview.net/forum?id=94t1GxmQkN>. OpenReview non-archival submission.
- [8] Douwe Kiela, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, Grusha Prasad, Amanpreet Singh, Pratik Ringshia, Zhiyi Ma, Tristan Thrush, Sebastian Riedel, Zeerak Waseem, Pontus Stenetorp, Robin Jia, Mohit Bansal, Christopher Potts, and Adina Williams. Dynabench: Rethinking benchmarking in NLP. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4110–4124. Association for Computational Linguistics, 2021. URL <https://aclanthology.org/2021.naacl-main.324/>.
- [9] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905, 2024.
- [10] Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. Reinforcement learning on web interfaces using workflow-guided exploration. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=ryTp3f-0->.
- [11] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, et al. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183, 2025.
- [12] Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J Pal, and Siva Reddy. AgentRewardBench: Evaluating automatic evaluations of web agent trajectories. *arXiv preprint arXiv:2504.08942*, 2025.
- [13] Qianou Ma, Dora Zhao, Xinran Zhao, Chenglei Si, Chenyang Yang, Ryan Louie, Ehud Reiter, Diyi Yang, and Tongshuang Wu. SPHERE: An evaluation card for human-ai systems, 2025. URL <https://arxiv.org/abs/2504.07971>.

- [14] Charles F. Manski. *Partial Identification of Probability Distributions*. Springer, 2003.
- [15] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229. ACM, 2019. doi: 10.1145/3287560.3287596. URL <https://arxiv.org/abs/1810.03993>.
- [16] OpenAI. Introducing SWE-bench Verified. Blog post, 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>. Accessed: 2026-05-01.
- [17] OpenAI. Why SWE-bench verified no longer measures frontier coding capabilities. <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>, 2026. Published February 23, 2026.
- [18] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program. *Journal of Machine Learning Research*, 22(164):1–20, 2021. URL <https://www.jmlr.org/papers/v22/20-303.html>.
- [19] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. URL <https://arxiv.org/abs/2307.16789>.
- [20] Christopher Rawles, Sarah Clinckemaulle, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents, 2025. URL <https://arxiv.org/abs/2405.14573>.
- [21] Lin Shi, Chiyu Ma, Wenhua Liang, Xingjian Diao, Weicheng Ma, and Soroush Vosoughi. Judging the judges: A systematic study of position bias in llm-as-a-judge, 2024. URL <https://arxiv.org/abs/2406.07791>.
- [22] Sierra Research. τ^3 -Bench: Advancing Agent Benchmarking to Knowledge and Voice. Research release, 2026. URL <https://sierra.ai/resources/research/tau-3-bench>. Accessed: 2026-04-30.
- [23] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents, 2024. URL <https://arxiv.org/abs/2407.18901>.
- [24] You Wang, Michael Pradel, and Zhongxin Liu. Are “solved issues” in SWE-bench really solved correctly? an empirical study. *arXiv preprint arXiv:2503.15223*, 2025.
- [25] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL <https://arxiv.org/abs/2404.07972>.
- [26] Frank F Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, et al. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2024.
- [27] Shuo Yang, Wei-Lin Chiang, Lianmin Zheng, Joseph E. Gonzalez, and Ion Stoica. Rethinking benchmark and contamination for language models with rephrased samples, 2023. URL <https://arxiv.org/abs/2311.04850>.
- [28] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022.

- 460 [29] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for
461 tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- 462 [30] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang,
463 Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica.
464 Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. URL <https://arxiv.org/abs/2306.05685>.
465
- 466 [31] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng,
467 Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic
468 web environment for building autonomous agents, 2023. URL <https://arxiv.org/abs/2307.13854>.
469
- 470 [32] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash
471 Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala
472 Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon,
473 Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy
474 Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks,
475 2025. URL <https://arxiv.org/abs/2507.02825>.

476 A Denominator Audit

Table 5: Denominator audit. Only infrastructure or pre-run failures are excluded before forming the completed evaluation set. Agent-caused aborts, invalid actions, and timeouts after task start stay in the denominator as native failures.

Domain	Attempted	Infra/pre-run excluded	Completed	Agent-caused fail	Notes
AGENTDOJO	FILL	FILL	300	FILL	Paired benign/injected arms remain linked at the case-unit level.
ANDROIDWORLD	FILL	FILL	300	FILL	Android app failures after task start remain in the evaluation set.
APPWORLD	FILL	FILL	300	FILL	Native database-state failures remain in the evaluation set.
τ^3 -bench retail	300	0	300	FILL	Tool misuse and policy-path failures remain in the evaluation set.
MINIWOB	FILL	FILL	300	FILL	Web UI task failures after task start remain in the evaluation set.
All domains	FILL	FILL	1500	FILL	Exclusions are not allowed to depend on whether the agent likely succeeded.

477 B Case Checklists and Scoring Inputs

478 Before scoring, we create one fixed case checklist per sampled case unit. The checklist is drafted
 479 with an LLM and then reviewed by two researchers in two cross-validation rounds. The draft records
 480 the user’s goal, what the official benchmark oracle or evaluator checks, what actions or state changes
 481 are needed for completion, and which stored artifacts would let a scorer decide the benchmark’s
 482 own claim. Every LLM-filled field must include either a rationale or a source pointer to the file
 483 and location from which the information was derived. If a draft item is unsupported, ambiguous, or
 484 stronger than the native claim, reviewers edit it or regenerate that item and review it again. Proposed
 485 stronger-measurement conditions are adjudicated separately and never folded into the native-aligned
 486 result without being marked. The checklist is locked before agent identities, agent traces, native
 487 scores, or evidence labels are shown to the scorer.

488 After each run, we collect the stored artifacts and score the record twice: the native benchmark
 489 evaluator gives the official/native label, and our scorer uses the locked checklist to assign Evidence
 490 Pass, Evidence Fail, or Unknown. Any LLM-assisted scoring judgment must also carry a rationale
 491 or source pointer. This separation is important: the paper reports both what the original benchmark
 492 would have counted and what the stored artifacts can support under the additional case checklist.

Table 6: Fields in the locked case checklist. The checklist is a scoring aid, not a new task.

Field	Purpose	Required support
User goal	Restates the task in benchmark language, including the intended object or recipient when available.	Task file, policy text, or rationale when the goal is synthesized across sources.
Native success claim	States what the released benchmark score claims to measure for this case.	Evaluator/oracle pointer, task text pointer, or policy pointer.
Official oracle or evaluator	Points to the official unit test, verifier, reward function, policy rule, or evaluator output used by the benchmark.	File path and code location, or released evaluator documentation.
Completion requirements	Lists the minimal state changes or side effects needed for the native claim to be true.	Source pointer for each requirement; rationale only when derived from multiple cited sources.
Evidence needed	Lists stored artifacts that can verify the claim, such as state diffs, database rows, receipts, verifier inputs, or unique identifiers.	Artifact schema pointer or rationale connecting the artifact to the native claim.
Evidence-label rule	Gives the scorer a deterministic rule for assigning Evidence Pass, Evidence Fail, or Unknown from stored artifacts.	Source pointer or rationale for each branch of the rule.
Stronger-measurement conditions	Separates reviewed non-native checks from the native-aligned result.	Brief rationale explaining why the condition goes beyond the native success criterion, plus reviewer adjudication.
Review notes	Records reviewer edits, regeneration reason, and final approval in two cross-validation rounds.	Reviewer id, round id, and decision note.

Table 7: Case-checklist drafting metadata. These values are filled from the scored manifest and released with the review package.

Field	Value
Drafter provider and model	OpenRouter; OpenAI GPT-5.4
Temperature / reasoning effort	0; high
Output budget / timeout	12,000 tokens; 180 seconds
Prompt version	FILL
Visible inputs	Task text; official policy, if any; evaluator code or evaluator description; database/API/browser/file/tool schema; trace schema; available post-run artifact types
Hidden inputs	Agent identity; agent trace; native score; outcome label; alternate-view verdicts
Human review rule	Two researchers cross-validate every checklist in two rounds before scoring. Unsupported requirements are edited or regenerated, then rechecked. Requirements that go beyond the benchmark claim are adjudicated separately and kept out of the native-aligned result.
Source hierarchy	Evaluator semantics first; task text and policy second; schema constraints needed to interpret evaluator-visible state third.

C Unknown-Reason Taxonomy

Each Unknown record receives one primary blocking reason. The labels make missing evidence actionable for benchmark authors; they are not a universal ontology.

Table 8: Unknown-reason labels. The priority rule assigns the reason that, if resolved first, would unblock the other missing evidence.

Reason	What is missing	Representative example
R1: Missing state query	The benchmark claim depends on a back-end or environment state value that was never queried or stored.	A database task has no post-run read of the relevant table.
R2: Unobservable side effect	The action is performed, but delivery, send, payment, receipt, or another side effect is not exposed in any artifact.	A message appears to be sent, but the tool returns no receipt or message id.
R3: Ambiguous identity mapping	Multiple records or recipients match the claim and the trace does not identify which one was acted on.	Two customers share the same name and no unique id is logged.
R4: Missing state-preservation evidence	The native task or policy requires preserving unrelated state, but no snapshot or diff can check it.	A calendar edit requires preserving other events, but no post-run diff is captured.
R5: Paired-arm asymmetry	A paired case requires both arms to be decidable, but one arm is decidable and the other is not.	The benign arm is verifiable while the injected arm’s security claim is not.
R6: Evaluator-output ambiguity	The released evaluator emits a label, but its stored inputs do not uniquely support that label under the benchmark claim.	A verifier accepts a string match without evidence that the structured target state was reached.
R7: Claim-scope mismatch	The trace supports a related task, but not the benchmark claim being reported.	The agent gives a helpful refund response, but the policy-relevant backend update required by the claim is absent.

496 D Additional Evaluation Tables

497 The main text keeps only the tables needed to follow the argument. This section holds the longer
498 tables that will be filled directly from the scored manifest.

Table 9: Human adjudication summary. Reviewed flags are records inspected after LLM-assisted scoring because they showed a native/evidence disagreement, Unknown label, stronger-measurement downgrade, or sampled audit trigger. Kept means the human audit kept the LLM-assisted substantive judgment. Corrections are scorer/checklist mismatches fixed before final aggregate reporting. The issue columns describe accepted benchmark-facing findings; they are not additional scores.

Benchmark	Reviewed	Kept	Corrections	Pending	Native eval. issues	Native evidence gaps	Stronger-only insights	Main audit lesson
ANDROIDWORLD	FILL	FILL						
τ^3 -bench retail	53	37	10	6	8	1	28	Action criteria, final-state semantics, and process constraints can be invisible in the scalar reward.
APPWORLD	FILL	FILL						
AGENTDOJO	FILL	FILL						
MINIWOB	FILL	FILL						
All benchmarks	FILL	FILL						

Table 10: Per-agent evidence-supported bounds by benchmark. Each cell reports native score, counted-only score, lower endpoint, upper endpoint, and width.

Benchmark	OpenAI GPT-5.4	Claude 4.7	DeepSeek V4 Pro
AGENTDOJO	FILL	FILL	FILL
ANDROIDWORLD	FILL	FILL	FILL
APPWORLD	FILL	FILL	FILL
τ^3 -bench retail	FILL	FILL	FILL
MINIWOB	FILL	FILL	FILL

Table 11: Stronger-measurement results. These rows apply only reviewed conditions beyond the benchmark’s native success criterion and are reported separately from Table 2.

Benchmark	Stronger condition family	E-Pass	E-Fail	Unknown	Coverage	Lower	Upper	Width	Difference from native-aligned result
AGENTDOJO	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
ANDROIDWORLD	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
APPWORLD	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
τ^3 -bench retail	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
MINIWOB	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL

Table 12: Descriptive case-resampling intervals for evidence-supported quantities. These intervals condition on stored traces and fixed agent versions; they do not include repeated-execution randomness, API drift, tool nondeterminism, or environment volatility.

Domain	Lower endpoint	Upper endpoint	Width	Counted-only score	Native score inside bounds
AGENTDOJO	FILL	FILL	FILL	FILL	FILL
ANDROIDWORLD	FILL	FILL	FILL	FILL	FILL
APPWORLD	FILL	FILL	FILL	FILL	FILL
τ^3 -bench retail	64.3–76.7%	64.7–77.0%	0.0–1.0%	64.4–76.9%	No
MINIWOB	FILL	FILL	FILL	FILL	FILL
All domains	FILL	FILL	FILL	FILL	FILL

Table 13: Full Unknown-reason distribution. The main text reports the top reason per benchmark; this table exposes the complete reason histogram.

Benchmark	R1	R2	R3	R4	R5	R6	R7	Interpretation
AGENTDOJO	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
ANDROIDWORLD	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
APPWORLD	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
τ^3 -bench retail	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
MINIWOB	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL

Table 14: Pairwise dominance margins for bound-separated comparisons. For a reported ordering $i > j$, the margin is $M_{i>j} = \text{LB}_{i,d} - \text{UB}_{j,d}$. The bootstrap interval is a descriptive stability check, not a formal significance test.

Domain	Pair	Bound decision	Margin and case-resampling interval	Interpretation
FILL FROM SCORED MANIFEST	–	–	–	Include every bound-separated pair and its case-cluster bootstrap dominance margin.

E Case-Level Audit Insights

The adjudication ledger records every flagged item, including records where human reviewers found that our checklist or scorer was too strict. Those internal corrections are not reproduced here. The appendix table below keeps only cases that remain substantively informative after correction: released benchmark/evaluator issues, evidence-retention gaps, and stronger-measurement blind spots that clarify what the native score does not claim. For τ^3 -bench retail, the case identifier in our sampled bundle is the released benchmark task id. The generated appendix table first maps each selected task reference to the corresponding official task before listing the audit insight.

Table 15: Map from selected appendix references to official τ^3 -bench retail tasks. In this bundle, the case id is the released benchmark task id; task summaries compress the official `reason_for_call` field.

Task ref.	Official user request, compressed	Native check, compressed	Insight family
T7	Exchange a water bottle and desk lamp; after confirmation, exchange only the desk lamp.	Exchange delivered items.	Confirmation timing.
T10	Return all items from two orders; if the cross-refund request is impossible, ask for human transfer.	Human transfer plus DB/NL reward.	Reward wiring conflict.
T18	Return a broken office chair, then switch to an exchange; if same item is unavailable, choose a gray leather chair.	Product lookup and exchange.	Same-item availability evidence.
T19	Return a water bottle and exchange two other items, or choose the option saving most money.	Return delivered items.	Partial-completion gap.
T25	Texas-shipped order: give tracking, return all except pet bed, refund to Amex, transfer if impossible.	Lookups only.	Native claim narrower than scenario.
T26	Texas-shipped order: give tracking, return all except pet bed, urgent request.	Returns plus human transfer.	Missing transfer ignored.
T27	Return hose/backpack and exchange hiking boots; if only one action is possible, prefer exchange.	Product lookup and exchange.	Confirmation timing.
T30	Damaged tablet request involving tracking, exchange/return, charger cancellation, and sneaker return.	Returns and cancellation.	Confirmation timing.
T33	Work-from-home order: if partial cancellation is impossible, keep the order but change default address to Seattle without revealing it.	Modify user address.	Sensitive-address disclosure.
T34	Work-from-home order: if partial cancellation is impossible, change pending-order address to New York.	Modify pending-order address.	Missing post-state evidence.
T35	Return a water-sensitive speaker and modify a laptop to a preferred 13-inch option.	Return plus pending-order modification.	Confirmation timing.
T36	Pending order exceeds credit limit; if needed, switch all items to cheapest options or cancel the order.	Modify pending-order items.	Object-semantics mismatch.
T50	Urgently undo a canceled order and receive all original items as soon as possible.	Human transfer.	Missing transfer ignored.
T71	Change an order address, modify items, then change payment preference at confirmation time.	Modify address and items.	Intermediate side effects.
T78	Change address, exchange a makeup kit, and cancel another order.	Address/item modification and cancellation.	Confirmation timing.
T86	Exchange a fleece jacket and change default address to a hidden Washington DC address stored in another order.	Modify item and user address.	Hidden-address process.
T91	Return skateboards and other items, with an exchange fallback for an e-reader.	Return and exchange.	Confirmation timing.
T97	Change a Los Angeles order to a hidden New York address and exchange a speaker.	Modify address and item.	Confirmation timing.
T101	Change a watch order address/items and modify another order's air purifier.	Multiple pending-order modifications.	Confirmation timing.
T102	Change a watch order to a hidden New York address, modify the watch, and exchange an air purifier.	Address/item modification plus exchange.	Hidden-address process.
T103	Return received items, change a pending order address/item, and get tracking for a canceled order.	Returns plus address/item modification.	Confirmation timing.
T105	Exchange two tea kettles to two distinct requested variants.	Exchange delivered items.	Attempt accepted as completion.
T112	Modify a laptop order to a hidden NYC address, change the laptop, and change a watch variant.	Multiple pending-order modifications.	Hidden-address process.

Table 16: Selected τ^3 -bench retail case-level insights after human audit. In the first column, G = GPT-5.4, C = Claude 4.7, D = DeepSeek V4 Pro, and “all” means all three models. We omit records whose only issue was our own scorer/checklist misalignment; those are corrected before final aggregate reporting and are tracked in the released adjudication ledger.

Official task(s)	Layer	Insight	What the retained artifacts show	Where the problem occurs
T50/C	Native false success	A required human-transfer action is absent, but the released evaluator still reports success.	The official action criterion requires <code>transfer_to_human_agents</code> ; <code>results.json</code> records <code>action_match=false</code> , and the retained task log contains no transfer call.	The scalar reward ignores the failed action check and is computed from other reward bases.
T26/all	Native false success	The same missing-transfer pattern appears across all three models.	The runs perform parts of the retail task, but the required transfer call is missing while the released reward remains success.	The evaluator has an action criterion that is not enforced by the reported score.
T105/D	Native false success	An attempted exchange is accepted as if the exchange completed.	The required exchange tool call does not match, and the retained order state has no completed exchange fields. The native natural-language assertion nevertheless accepts the attempt.	The evaluator conflates attempted assistance with the state change required by the task.
T10/G	Native false / wiring conflict	The task text and action evidence support “do what can be done, then transfer,” but the scalar reward fails the run.	The log shows return calls for both orders followed by a successful transfer. Released action checks are satisfied, but <code>db_match=false</code> drives <code>reward=0</code> .	The DB oracle conflicts with the task scenario and action criteria.
T36/G+D	Native false-success candidate	Item identifiers can match while the post-state object semantics look wrong.	The write call uses the listed item ids and replacement ids, but the returned order contains implausible product/object fields, such as non-camera rows inheriting T-shirt-like options.	The state checker appears to validate target ids without checking product semantics.
T34/C	Native evidence gap	The retained artifacts do not independently verify the final database state.	The trace contains a successful <code>modify_pending_order_address</code> on a tool return rather than a post-run DB snapshot for the target order.	Evidence retention relies on a tool return rather than an authoritative post-run state read.
T25/C	Stronger-only design gap	The user asks for human transfer after Amex refund is impossible, but this requirement is absent from the official scoring criteria.	The scenario says to transfer if the agent cannot help; the user asks twice for a human, and the agent refuses. The official criteria require only lookups and therefore can still report success.	The benchmark’s native claim is narrower than the user scenario.
T102/C; T86/G+D; T112/all	Stronger-only privacy/process gap	Stored-address tasks can pass final-state checks even when the agent asks the user to reveal, or itself reveals, the address.	The native evaluator checks final DB updates. The retained transcripts show agents requesting the hidden address or printing it back to the user.	A final-state score misses privacy and process constraints around how the state was obtained.
T18/G+C	Stronger-only evidence gap	Same-item unavailability is either undecidable or contradicted by retained product evidence.	One run lacks enough evidence to decide whether a same-item exchange was unavailable; another has product evidence contradicting the claimed unavailability.	The stronger claim depends on product-availability evidence that the native score does not require.
T19/G+C	Stronger-only completion gap	A quote or partial return can satisfy the native view while a requested exchange remains incomplete.	The retained evidence supports the native outcome, but the stronger review finds no completed exchange update for the requested items.	The native claim does not distinguish partial assistance from completing the user’s broader exchange request.
T33/C	Stronger-only disclosure gap	The agent reveals an address while still satisfying the native final-state claim.	The transcript prints the Seattle address to the user, violating the reviewed no-disclosure condition.	The native evaluator checks state outcome, not sensitive-information exposure.
T71/C	Stronger-only side-effect gap	The final state can pass even after repeated writes to the same order.	The run contains two modify-order calls for the same order.	Final-state checking does not expose unnecessary or risky intermediate side effects.
T77/G; T27/D; T30/D; T35/C; T78/D; T91/D; T97/D; T101/D; T103/G+D	Stronger-only confirmation-timing gap	Many runs update state before a separate, explicit user confirmation for the specific write.	The traces show DB-updating calls in the same turn as a summary, after combined approval, or before a fresh post-summary “yes.”	Final-state evaluators can miss consent timing even when the final database state is correct.

F Reproduction Setup

The released package separates checklist drafting, benchmark execution, and evidence scoring. Paths in the table are release-relative; hostnames, live API keys, and machine-specific absolute paths are not part of the public artifact.

Table 17: End-to-end reproduction pipeline. The adapter stage preserves raw benchmark evidence and native evaluator outputs; final Evidence Pass, Evidence Fail, and Unknown labels are assigned only in the scoring stage.

Stage	Reproduction details
Checklist draft	Entry point: <code>make draft</code> . Model/configuration: OpenRouter OpenAI GPT-5.4, temperature 0, high reasoning effort, 12,000-token output budget, 180-second timeout. Input: <code>case_packet.md</code> . Outputs: <code>checklist.yaml</code> , API response, LLM-call log, and reasoning summary.
Validation and locking	Entry point: offline checklist validator and case-lock update script. Inputs: <code>case_packet.md</code> and draft checklist. Outputs: validated checklist and case-lock record.
Evidence collection	Entry point: benchmark-specific adapter over the official benchmark runner. Configuration: agent settings from <code>configs/agents.yaml</code> ; no final evidence verdict. Inputs: <code>job.json</code> , benchmark installation, official assets, and agent config. Outputs: raw run record, artifact manifest, raw artifacts, native evaluator output, stdout/stderr, and call logs.
Evidence scoring	Entry point: <code>make score</code> . Model/configuration: Codex CLI GPT-5.4, xhigh reasoning effort, read-only sandbox, at most two attempts. Inputs: locked checklist and saved evidence directory. Outputs: score JSON/YAML, score manifest, stdout/stderr logs, event stream, telemetry, and reasoning summary.

Table 18: Model-call configuration for the main experiment.

Model	Provider	Temp.	Output budget	Timeout / retry
OpenAI GPT-5.4	OpenRouter	0	4096	120 s / 2
Claude Opus 4.7	OpenRouter	0	4096	120 s / 2
DeepSeek V4 Pro	OpenRouter	0	4096	120 s / 2

Table 19: Machine roles used for raw evidence collection. Drafting and final scoring can run on any authenticated CPU machine and require no GPU.

Benchmarks	Machine role	Configuration	Docker
AGENTDOJO, APPWORLD, τ^3 -bench retail, MINIWOB	General adapter VPS	8 CPU, 32 GB memory, no GPU	Yes
ANDROIDWORLD	Mobile-app adapter VPS	8 CPU, 32 GB memory, no GPU	Yes

G Release and Result Files

Public release should make denominators and bounds auditable without exposing live credentials, real third-party keys, or side-effecting services.

- **Released publicly.** Case ids, locked case checklists, native claim, checklist rationales or source pointers, stronger-measurement flags and rationales, native label, evidence label, reason code, bound contribution, denominator audit, aggregate tables, plotting scripts, and audit instructions.
- **Released under access control when needed.** Full traces that contain prompt-injection content, synthetic personal data, or credential-shaped strings after scrubbing and sandbox re-pinning.
- **Not released.** Live credentials, real API keys, or artifacts that would reproduce side effects against non-sandboxed services.
- **Versioning.** Reported bounds are paired with a version tuple for the scorer, manifest, checklist template, checklist-generation prompt, and Unknown-reason taxonomy.

525 The final build should be produced from the scored manifest. The file
526 `outputs/latex/results_macros.tex` should set `\resultdatatrue` and define the domain-
527 level macros used in Tables 2–4. Optional row files can populate the longer appendix tables. Missing
528 values should remain visibly marked as `FILL` rather than replaced by guessed numbers.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes].

Justification: The abstract and introduction state the reporting method, the fixed four-domain study, and the intended scope. The discussion explicitly limits the claims to evidence-supported reporting rather than benchmark validity or a new leaderboard.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes].

Justification: The Discussion section lists limitations of the locked claim, taxonomy scope, three-agent/four-benchmark design, artifact-level Unknown labels, and bootstrap interpretation.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A].

Justification: The paper defines a reporting object and counting rule but does not present formal theorem statements or proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes].

Justification: The Method, Experiments, Evaluation, and appendix sections describe case-checklist locking, denominator rules, audit checks, result files, and how the reported quantities are produced from the released manifests and scoring script.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in

some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes].

Justification: The paper states that manifests, locked case checklists, checklist rationales or source pointers, reason labels, audit records, robust-ranking summaries, and a scoring script are released; the Release and Result Files appendix specifies which artifacts are public and which are access-controlled.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes].

Justification: The paper specifies domains, case-unit selection, agents as fixed probes, denominator handling, case-checklist drafting and locking, resampling, audit, and rerun checks.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes].

Justification: The paper reports case-cluster bootstrap intervals as descriptive resampling diagnostics and explains their assumptions and limitations in Sections 3 and 3.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No].

Justification: The current manuscript does not yet give full hardware, memory, wall-clock execution, or total compute details for every benchmark run.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes].

Justification: The work uses public or benchmark-provided artifacts and includes release safeguards for traces, synthetic personal data, credentials, and access-controlled materials.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes].

Justification: The Discussion and Release and Result Files appendix describe benefits for benchmark reporting and risks around state-changing workflows, security-related traces, synthetic personal data, and operational details.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [Yes].

Justification: The Release and Result Files appendix separates fully public artifacts, access-controlled traces, and non-released credentials or operational artifacts.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No].

Justification: The manuscript cites the existing benchmarks used in the study, but the current draft does not yet enumerate every asset license and terms-of-use constraint in one place.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes].

Justification: The paper documents the released manifests, case checklists, checklist rationales or source pointers, reason labels, denominator audits, scoring script, result files, and versioning in the Release and Result Files appendix.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A].

Justification: The blinded audit is an expert audit of benchmark records and evidence labels, not a crowdsourcing experiment or a study of human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

844 Answer: [N/A].

845 Justification: The work does not collect personal data from human subjects or run a human-

846 subjects experiment; the audit concerns benchmark traces and evidence-label decisions.

847 Guidelines:

- 848 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
- 849 with human subjects.
- 850 • Depending on the country in which research is conducted, IRB approval (or equivalent)
- 851 may be required for any human subjects research. If you obtained IRB approval, you
- 852 should clearly state this in the paper.
- 853 • We recognize that the procedures for this may vary significantly between institutions
- 854 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
- 855 guidelines for their institution.
- 856 • For initial submissions, do not include any information that would break anonymity (if
- 857 applicable), such as the institution conducting the review.

858 **16. Declaration of LLM usage**

859 Question: Does the paper describe the usage of LLMs if it is an important, original, or

860 non-standard component of the core methods in this research? Note that if the LLM is used

861 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

862 scientific rigor, or originality of the research, declaration is not required.

863 Answer: [Yes].

864 Justification: The paper describes the LLM-assisted case-checklist drafter and scorer, their

865 inputs, the rationale/source-pointer requirement, and two-round human review before locked

866 checklists are used for scoring.

867 Guidelines:

- 868 • The answer [N/A] means that the core method development in this research does not
- 869 involve LLMs as any important, original, or non-standard components.
- 870 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
- 871 be described.